

3 公钥密码和 PKI

3.1 证书颁发机构(Certificate Authority,CA)

证书颁发机构（CA）是负责签发数字证书的可信实体，其颁发的证书能够验证公钥与命名主体之间的所有权关系。目前，VeriSign等商业CA作为根CA运营，用户需支付费用才能获取其签发的证书。然而，我们也可以自行创建根CA并自主签发证书，无需依赖商业CA。具体操作中，根CA的证书采用自签名方式，不同于普通证书由上级CA签发。这类根证书通常被预置在操作系统、浏览器等PKI依赖环境中，享有无条件信任。使用OpenSSL创建证书时，需准备.cnf格式的配置文件，该文件被ca、req和x509等命令调用。建议从/usr/lib/ssl/openssl.cnf获取模板，复制到工作目录后，需根据配置文件[CA default]部分的指引创建相应子目录结构：

```
dir          = ./demoCA          #Where everything is kept
certs        = $dir/certs        #Where the issued certs are kept
crl_dir      = $dir/crl          #Where the issued crl are kept
new_certs_dir = $dir/newcerts    #Default place for new certs
database     = $dir/index.txt    #Database index file
serial       = $dir/serial       #The current serial number
```

在配置CA环境时，index.txt文件只需通过touch命令创建空文件即可，而serial文件则需要包含一个字符串格式的起始序列号（例如“1000”）。完成openssl.cnf配置文件的设置后，即可开始创建和颁发证书。需要注意的是，serial是一个纯文本文件，用于记录证书的序列号。作为证书颁发机构（CA），我们需要首先生成一个自签名的根证书。由于根CA是信任链的起点，其证书不由其他CA签发，而是自行签名，因此被视为完全可信。该证书将作为根证书，用于后续签发其他证书。可以通过OpenSSL命令生成CA的自签名证书：

```
% sudo openssl req -new -x509 -keyout ca.key -out ca.crt -config
openssl.cnf
```

在生成自签名根证书的过程中，系统会提示你输入必要的信息（如国家名称、通用名称等）并设置密码。请务必妥善保管该密码，因为后续每次使用该CA为其他实体签发证书时都需要验证此密码。生成的密钥和证书将分别保存在`ca.key`和`ca.crt`两个文件中——其中`ca.key`包含CA的私钥（需严格保密），而`ca.crt`则是包含公钥的证书文件，用于公开分发和信任链建立。



3.2 为 PKILabServer.com 生成证书

作为自建的根CA，我们可以为客户签发数字证书。以PKILabServer.com公司为例，该公司需要完成三个步骤来获取数字证书：首先，PKILabServer.com需要生成自己的RSA公私钥对。这可以通过执行OpenSSL命令实现，生成的密钥将被加密存储在server.key文件中。执行命令时，系统会提示设置密码以保护私钥安全：

```
% sudo openssl genrsa -des3 -out server.key 1024
```

在生成密钥对后，PKILabServer.com需要创建证书签名请求(CSR)。这个步骤将公司的公钥和身份信息(如组织名称、地理位置等)打包成一个标准请求文件。特别需要注意的是，必须使用“PKILabServer.com”作为通用名称(Common Name)，这将成为证书验证时的重要标识。生成的CSR文件将被提交给CA机构，CA在核实请求中的信息与服务器真实身份相符后，才会使用其根证书私钥对该请求进行签名：

```
% sudo openssl req -new -key server.key -out server.csr -config  
openssl.cnf
```

最后一步是由CA对CSR文件进行数字签名以生成正式证书。在实际应用场景中，企业通常需要将CSR提交给VeriSign等权威CA进行签名验证。而在本实验中，我们将利用自建的根CA来完成这一关键步骤——使用CA的私钥对server.csr进行签名，生成包含PKILabServer.com公钥信息的正式证书server.crt：

```
% sudo openssl ca -in server.csr -out server.crt -cert ca.crt -keyfile  
ca.key  
-config openssl.cnf
```

如果 OpenSSL 拒绝生成证书，通常是由于请求中的名称与 CA 的名称不匹配。具体匹配规则可在配置文件的[policy_match]部分查看。遇到这种情况时，您有两种解决方案：一是调整证书请求中的名称信息以符合现有策略要求；二是直接修改CA的签发策略。配置文件还包括另一个限制较少的策略(称为policy_anything)。你可以通过更改以下行来选择该策略：

```
"policy = policy_match" change to "policy = policy_anything".
```

3.3 网站使用 PKI

在本实验中，我们将研究网站如何利用公钥证书实现安全通信。我们选用PKILabServer.com作为实验域名。由于这是一个本地测试域名，我们需要在系统的/etc/hosts文件中添加一条解析记录，将该域名映射到本地回环地址

127.0.0.1。这样，当我们在浏览器中访问PKILabServer.com时，请求将被正确路由到本地服务器，为后续的HTTPS证书验证实验建立基础环境。

```
127.0.0.1 PKILabServer.com
```

接下来，让我们使用前面任务中生成的证书启动一个简单的web 服务器。OpenSSL 允许我们使用 s_server 命令启动一个简单的web 服务器：

```
# Combine the secret key and certificate into one file
% cp server.key server.pem
% cat server.crt >> server.pem
# Launch the web server using server.pem
% openssl s_server -cert server.pem -www
```

当通过 `https://PKILabServer.com:4433/` 访问服务器时，浏览器（如Firefox）会显示安全证书错误提示：“pkilabserver.com:4433 uses an invalid security certificate. The certificate is not trusted because the issuer certificate is unknown”。这是因为我们使用的是自建CA（ca.crt）签发的证书，而非浏览器预置信任的权威CA（如VeriSign）颁发的证书。要让Firefox信任这个自签名证书，有两种解决方案：

向Mozilla申请将CA证书加入Firefox信任库：这是VeriSign等商业CA的常规做法，但由于我们的自建CA缺乏广泛的市场认可度，此方案不可行。

手动导入CA证书：用户可以将ca.crt加载到Firefox中，通过点击以下菜单序列手动添加CA的证书到Firefox浏览器中：

```
Edit -> Preference -> Advanced -> Certificates -> View Certificates.
```

你将看到一个已被Firefox接受的证书列表。从这里我们使用“import”导入我们自己的证书。请导入ca.crt，并选择以下选项：“Trust this CA to identify web sites”。你将看到我们的CA证书在Firefox接受证书的列表中。

现在，将浏览器指向`https://PKILabServer.com:4433`。请描述并解释你的观察结果。请同时完成以下任务：

1. 修改server.pem的一个字节，并重新启动服务器，重新加载URL。你观察到什么？最后确保你能恢复了原始server.pem。
2. 由于PKILabServer.com指向本地主机，如果使用<https://localhost:4433>代替，我们将连接到相同的web服务器。请执行该操作，描述和解释你的观察。

3.4 性能比较：RSA 和 AES



在本任务中，我们将研究公钥算法的性能。请准备一个包含16字节的文件(message.txt)，同时生成1024位的RSA公钥/私钥对。然后，执行以下步骤：

1. 使用公钥加密 message.txt，将输出结果保存到 message_enc.txt 中。
2. 使用私钥解密 message_enc.txt。
3. 使用128位AES密钥对 message.txt 进行加密。
4. 比较上面每个操作所花费的时间，并描述你的观察结果。(如果一个操作太快，可以重复5000次，然后取平均值)。

完成上述练习后，现在可以使用 OpenSSL 的 speed 命令进行这样的基准测试。请描述你的观察结果是否与 speed 命令的输出结果相似。

下面的命令显示了使用速度对 rsa 和 aes 进行基准测试的示例：

```
% sudo openssl speed rsa
% sudo openssl speed aes
```

3.5 创建数字签名

在本任务中，我们将使用 OpenSSL 生成数字签名。请准备一个任意字节大小的文件 example.txt，同时生成RSA公钥/私钥对。请按照以下步骤进行操作：

1. 对 example.txt 的 SHA256 哈希值签名，将输出保存为 example.sha256。
2. 验证数字签名： example.sha256。
3. 稍微修改 example.txt，并再次验证数字签名。

请描述你是如何完成上述操作的(例如你使用了什么命令等)。根据你的观察，请解释为什么数字签名是有用的。